

Microcontrollers and Circuits

Authors: Avi Bhadore, Hailey Doleweerd, Kai Collins, Punit Shah, Karnan Tamilchelvan,
Eric You, Benjamin Zaionz, Michelle Chen, Michael McPhee, Anna Chen, Silas Ifeanyi,
Nancy Nelson, Chris Rennick

Table of Contents

Contents

Table of Contents	2
Introduction	4
Outcomes	4
Required Materials and Software.....	4
Resources	5
Part 1- Building and Testing a Simple Circuit.....	5
Outcomes.....	5
Background.....	5
Breadboards.....	5
Wiring Conventions	6
LED Polarity	7
Activity 1- LED Circuit.....	7
Multimeters	9
Part 2 – Microcontrollers.....	9
Outcomes.....	10
Background.....	10
Arduino Uno Board	10
Arduino Pinout Map.....	10
Pull-up and Pull-down Resistors.....	11
Activity 2 - Arduino-controlled LED Circuit.....	12
Arduino Integrated Development Environment (IDE)	12
Modify your Circuit	13
Create Your Project (Sketch).....	15
Add your Code	16
Test your Circuit	17

Part 3 – Analog Inputs and UART	17
Outcomes	17
Background.....	18
Analog Signals	18
Measuring Analog Signals using Microcontrollers – ADCs.....	18
Potentiometers	18
UART and Serial Communication	18
Activity 3 – Read and Display Analog Values	19
Wire your Circuit	19
Add your Code	20
Test your Circuit	20

Introduction

This session is an introduction to both electronic circuits and microcontrollers. You'll start by using common tools and components to build and test simple circuits. Then you'll begin learning about microcontrollers, enough to control simple circuits.

Outcomes

You should be able to do the following things by the end of this session:

- Breadboard a simple circuit given a circuit diagram
- Verify proper operation of the circuit
- Use a multimeter to measure resistance and voltage
- Install and set up the microcontroller application
- Control simple circuits using a microcontroller

Required Materials and Software

- 2 Switches (Push Button)
- 2 LEDs
- 1 Breadboard
- 2 Resistors (220 Ω)
- 2 Resistors (10 k Ω)
- 1 Potentiometer
- 1 Multimeter
- 1 Arduino Uno
- Assorted Wires

Software:

Download and install the Arduino's Integrated Development Environment from the [official Arduino website](#). Follow the setup instructions that pop up when you open the file.

Resources

Building and Testing a Simple Circuit

- [Basic Electronics for Beginners in 15 Steps](#)
- [10 Basic Electronics Components and their functions](#)
- [How to Read a Schematic](#)
- [How To Use a Breadboard - Breadboard Basics](#)
- [How to Connect Arduino to a Breadboard \(2 min Setup!\)](#)
- [THE BEST Multimeter tutorial \(HD\)](#)

Building and Testing a Simple Circuit with MCUs

- [Short and sweet: pull-up/pull down resistors explained](#)

Analog Inputs and UART

- [Serial Communications Explained: UART, I2C, and SPI](#)
- [Easy Arduino Tutorial: #5 Analog Input](#)

Part 1- Building and Testing a Simple Circuit

In this section you'll build and test a simple circuit. In the process, you'll learn how to use a breadboard, interpret circuit diagrams, identify standard components, and use a digital multimeter.

Outcomes

You should be able to do the following things by the end of this section:

- Breadboard a simple sensor-based circuit given a circuit diagram
- Verify proper operation of the circuit

Background

Breadboards

A breadboard (see Figure 1 left) is a prototyping tool used for building and testing circuits without the need for soldering. It consists of rows and columns of conductive strips underneath a plastic surface with holes, allowing you to easily insert components and

wires to form electrical connections. A typical breadboard is divided into two main areas: the power rails and the terminal strips.

The power rails, running along the sides, are used for connecting power (e.g., 3.3V or GND) throughout your circuit. The terminal strips in the center are arranged in rows and connect vertically underneath the board, making it possible to link components without direct wiring. The right side of Figure 1 shows how the connections are made internally on a typical breadboard.

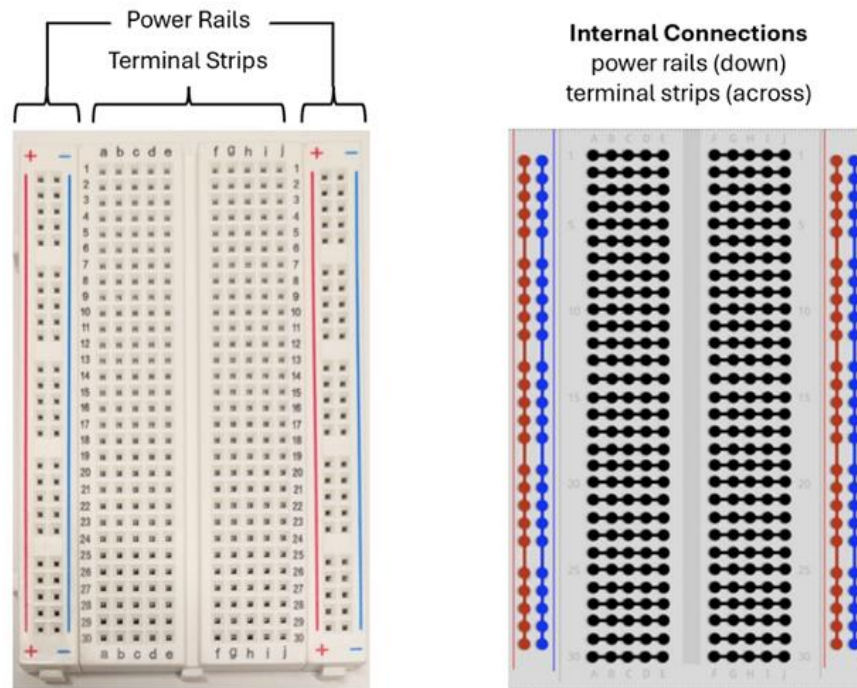


Figure 1 – Breadboard and its Internal Connections

Wiring Conventions

When wiring circuits or creating schematics, it's helpful to follow color conventions to make your work easier to read, debug, and share. A common standard is to use red wires for power (V_{cc}) and black wires for ground (GND).

Beyond that, it's up to you to define your own consistent system. A good practice is to use one wire color per signal or per section of a circuit, which keeps things organized, especially as circuits grow more complex.

When you're just starting out, it's okay to use a variety of colors to help visualize how different parts of the circuit connect and how signals flow. Over time, developing your own consistent wiring style will make building and troubleshooting much easier.

LED Polarity

Many electronic components, such as resistors, are non-polar. This means that the orientation of the component relative to the flow of electricity does not matter. Other components are polar, where the orientation of the component relative to the flow of electricity does matter. One such component is a diode. Diodes are designed specifically to allow electricity to flow in a certain direction and not in the other. LEDs (light emitting diodes) are diodes that also emit light. Figure 2 shows an LED.

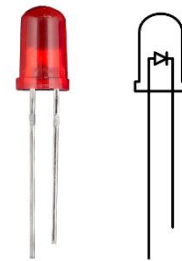


Figure 2 - LED

Notice that one leg is longer than the other. The longer leg is the anode and is connected to a positive voltage. The shorter leg is the cathode and is connected to a negative voltage (GND). If the LED is connected in the wrong orientation, then no electricity will flow, and the LED will not light up.

Activity 1- LED Circuit

In this activity you will control an LED with a push button (switch). When the button is pressed, the LED will turn on, and when the button is released, the LED will turn off. Figure 3 shows the circuit you're going to build.

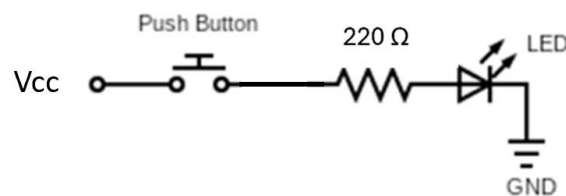


Figure 3: LED Circuit

This circuit has three components, a push button, a resistor, and an LED.

If you've never used a breadboard before, please refer to Figure 4 and follow these instructions:

1. Place the push button on the breadboard straddling the centre divider
2. Place the 220 Ω resistor (red, red, brown) on the breadboard with each leg in a different unused row
3. Place the LED on the breadboard with each leg in a different unused row
4. Connect the components
 - a. Connect one of the pushbutton's legs to the breadboard's power rail with a red wire
 - b. Connect its other leg to the closer leg of the resistor
 - c. Connect other leg of the resistor to the LED's anode (longer leg)
 - d. Connect the LED's cathode (shorter leg) to the breadboard's ground rail (black wire)
5. Power the circuit (using the Arduino Uno)
 - a. Connect the breadboard's power rail to the 5V pin on the Arduino with a red wire
 - b. Connect the breadboard's ground rail to the Arduino's GND pin with a black wire
6. Double check your connections
 - a. Arduino's 5V to the power rail
 - b. Breadboard's power rail to the pushbutton
 - c. Pushbutton to the 220 Ω resistor
 - d. Resistor to the LED's anode (longer leg)
 - e. LED's cathode (shorter leg) to the breadboard's ground rail
 - f. Ground rail to Arduino's GND
7. Connect your Arduino to your computer with a USB cable
8. Test your circuit
 - a. Push the button ... LED should come on
 - b. Release the button ... LED should go off

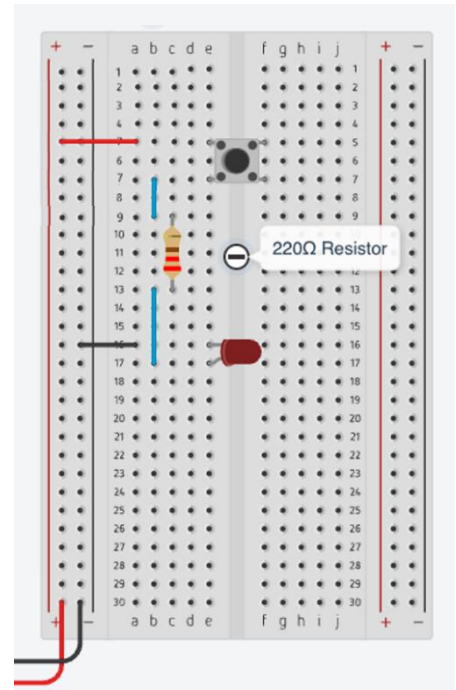


Figure 4: Wired Breadboard

Multimeters

A multimeter is a valuable tool for debugging and exploring how electricity flows in a circuit. Whether you're using a handheld multimeter or a bench/tabletop multimeter, the basic functions are the same. Here's how to measure resistance and voltage in this project:

Measuring Resistance

1. Power off your Arduino.
2. Disconnect one leg of the resistor from the breadboard to avoid interference from the circuit.
3. Set the dial to the Ω (ohm) symbol.
4. Touch one probe to each side of the resistor (see Figure 10)
5. You should see a reading close to 220 Ω .
6. Record your reading. Remember to include the appropriate units ... $R =$

Tip: Handheld multimeters may require manual range setting, while tabletop multimeters often auto-range and may give more precise decimal values.

Measuring Voltage

1. Keep the circuit powered by ensuring the Arduino is powered.
2. Set the dial to DC Voltage ($V=$).
3. Place the black (COM) probe on GND.
4. Place the red probe on the LED's anode (the longer leg).
5. When the button is pressed, you'll see ~2V across the LED (its voltage drop)
6. Record your reading (with units) ... $V_{LED} =$

Part 2 – Microcontrollers

Microcontrollers are small one-chip computers that are at the core of much of today's technology to automate and control tasks. Generally, they have a Central Processing Unit (CPU), some memory, and programmable input and output ports. In this activity, you'll get

hands-on experience with the Arduino Uno, a simple microcontroller equipped with input and output pins. These pins let you connect components such as sensors, motors, and LEDs. Through these connections, the Arduino can receive data from connected devices as well as send instructions back to them. You'll learn how to build and control digital and analog circuits to transmit and receive data signals between devices, using wiring components and debugging tools.

Outcomes

You should be able to do the following things by the end of this section:

- Install and set up the microcontroller application
- Control a simple sensing circuit using a microcontroller

Background

Arduino Uno Board

The Arduino UNO (see Figure 5) is a microcontroller board based on the ATmega328P. It has 14 digital input/output pins and 6 analog inputs. It's an easy-to-use system designed for hobbyists.



Figure 5: Arduino Uno

Arduino Pinout Map

A pinout map shows the available pins on your Arduino Uno microcontroller and what functions those pins can be used for. Each pin can have multiple uses:

- Digital I/O (Input/Output) allows the Arduino Uno to **read and write** to components (LEDs, buttons, etc.) via digital signals. Digital output pins can send signals, either

HIGH (5V) or LOW (0V), to other components. Digital input pins can read the state of a digital signal, such as whether a button is pressed or not. In this activity you'll use a digital output to turn an LED on and off, and a digital input to detect whether a button is pressed.

- Analog inputs **read** continuous voltage from sensors or potentiometers.
- Communication protocols allow the Arduino Uno to **communicate** with peripherals via I2C, SPI and UART. Additional info on these protocols is available in section 2.

Figure 6 shows the pinout map for the Arduino Uno board you're using in this activity. You will likely need to refer to this figure throughout the activity. The digital I/O pins are labeled D0 – D15, and the analog inputs are labeled A0 – A5. There are also pins that are dedicated to the communication protocols labeled I2C, SPI and UART.

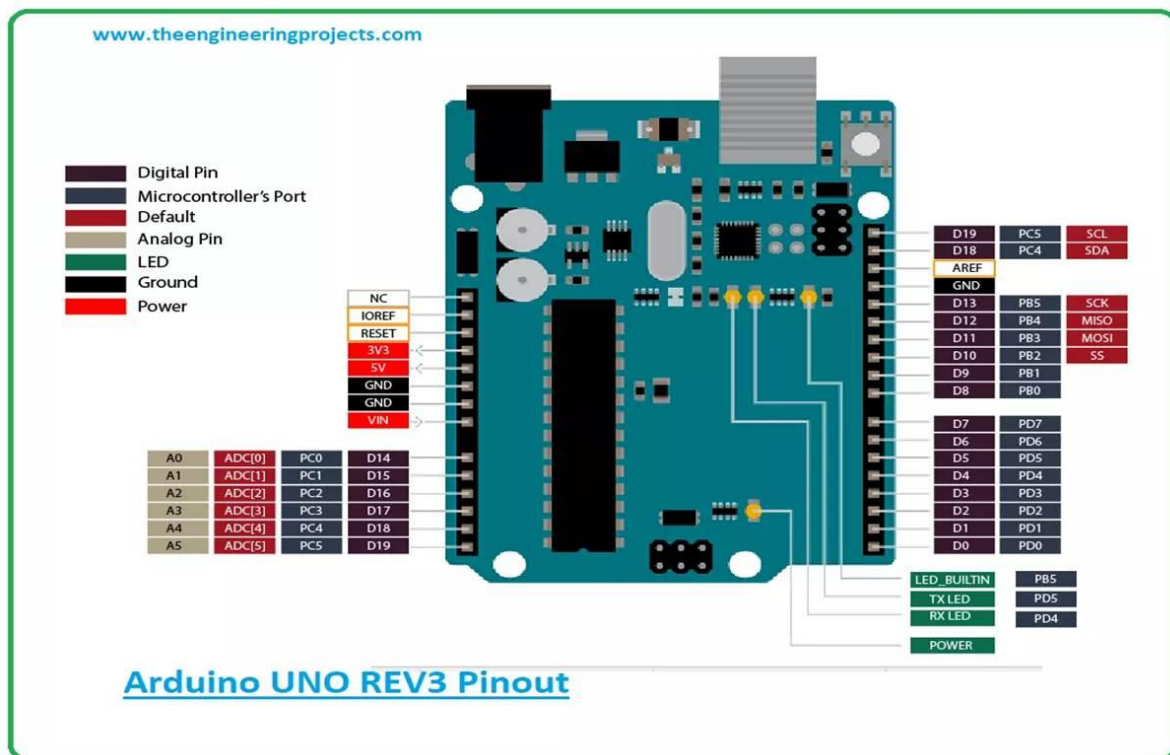


Figure 6: Arduino Uno Pinout Map

Pull-up and Pull-down Resistors

If a line in a circuit isn't explicitly set to a voltage it is considered "floating". In electronics we don't want this, so we use pull-up and pull-down resistors to passively set the voltage of lines that are not being set otherwise. *Figure 7* shows the wiring diagram for pull-up and pull-down resistors.

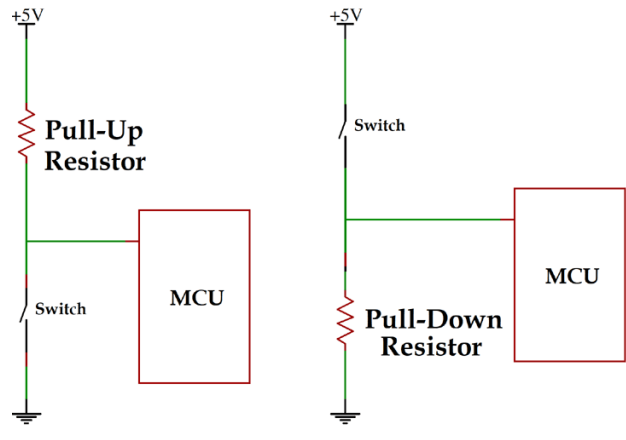


Figure 7: Pull-up and Pull-down Resistors

Consider the pull-down resistor diagram. When the switch is open, the voltage across the device is very close to ground (if the pull-down resistor has a low resistance, which they typically do). When the switch is closed, the voltage on the device is 5V, and the resistor measures 5V across. This behaviour is mirrored for the pull-up resistor.

Activity 2 - Arduino-controlled LED Circuit

Arduino Integrated Development Environment (IDE)

1. If you haven't installed the Arduino's IDE yet, download it from the [official Arduino website](https://www.arduino.cc/en/main/software). Follow the setup instructions that pop up when you open the file.
2. Start the application and do the following:
 - a. **Select Tools > Board > Boards Manager**
 - b. Search for "Arduino UNO R4"
 - c. Select **Install** and accept all prompts for installation
 - d. Once the installation is complete, please confirm you see the following
"Tools > Board> Arduino UNO R4 Boards > Arduino UNO R4 Minima

Modify your Circuit

Integrating the Arduino provides a centralized way to control and automate the system. It acts as a small computer or “control center,” processing inputs, making decisions based on code, and sending commands to connected components. Figure 8 shows the modifications you’ll need to make to your circuit. D2 and D3 are Arduino digital I/O ports, and a pull-down resistor is added to the push button circuitry to ensure its resting state is LOW.

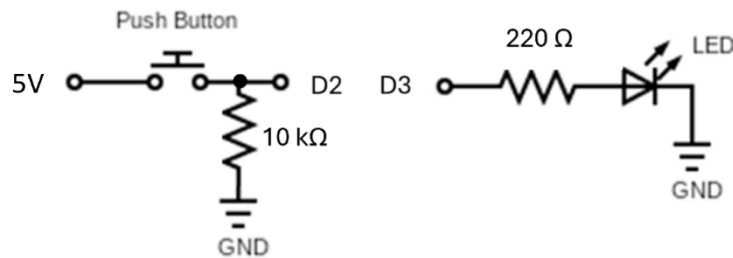


Figure 8: Modified LED circuit

A helpful way to understand this setup is to think of it as two separate circuits connected to the Arduino. The first involves the push button, which sends a signal to the Arduino when pressed. The second controls the LED, which is powered by the Arduino. When the Arduino receives the button input, it activates the second circuit.

Here’s a more detailed description of how the two parts of the circuit work:

The push button is connected between the 5V power rail and a digital input pin on the Arduino (e.g., pin 2). When the button is pressed, the circuit closes and the input pin is directly connected to 5V, which the Arduino reads as “HIGH.” When the button is not pressed, the input pin is connected to ground through a 10kΩ pull-down resistor. This resistor lets current to pass through to GND when the button is not pressed, which prevents the pin from floating and ensures a reliable “LOW” reading.

The LED is connected in series with a current-limiting resistor to prevent it from burning out. The resistor connects to a digital output pin, which can be toggled HIGH or LOW to turn the LED on or off.

Refer to Figure 9 for a sample layout of the modified circuit.

1. Remove the wire connecting the push button to the 220 Ω resistor
2. Connect that leg of the push button to a 10 kΩ resistor (brown, black, orange)

3. Connect the resistor's other leg to the breadboard's ground rail with a wire (preferably a black one!)
4. Connect the circuit to the Arduino Uno
 - a. Connect the pushbutton lead with the 10 k Ω to digital pin 2 (there are now 2 connections in the row)
 - b. Connect the free leg of the 220 Ω resistor to digital pin 3 using a wire
5. Double check your connections
 - a. Arduino's 5V to the power rail
 - b. Breadboard's power rail to the pushbutton
 - c. Pushbutton to 10 k Ω resistor and to digital pin 2
 - d. Other end of 10 k Ω resistor to breadboard's ground rail
 - e. Digital pin 3 to the 220 Ω resistor
 - f. 220 Ω resistor to the LED's anode (longer leg)
 - g. LED's cathode (shorter leg) to the breadboard's ground rail
 - h. Ground rail to Arduino's GND

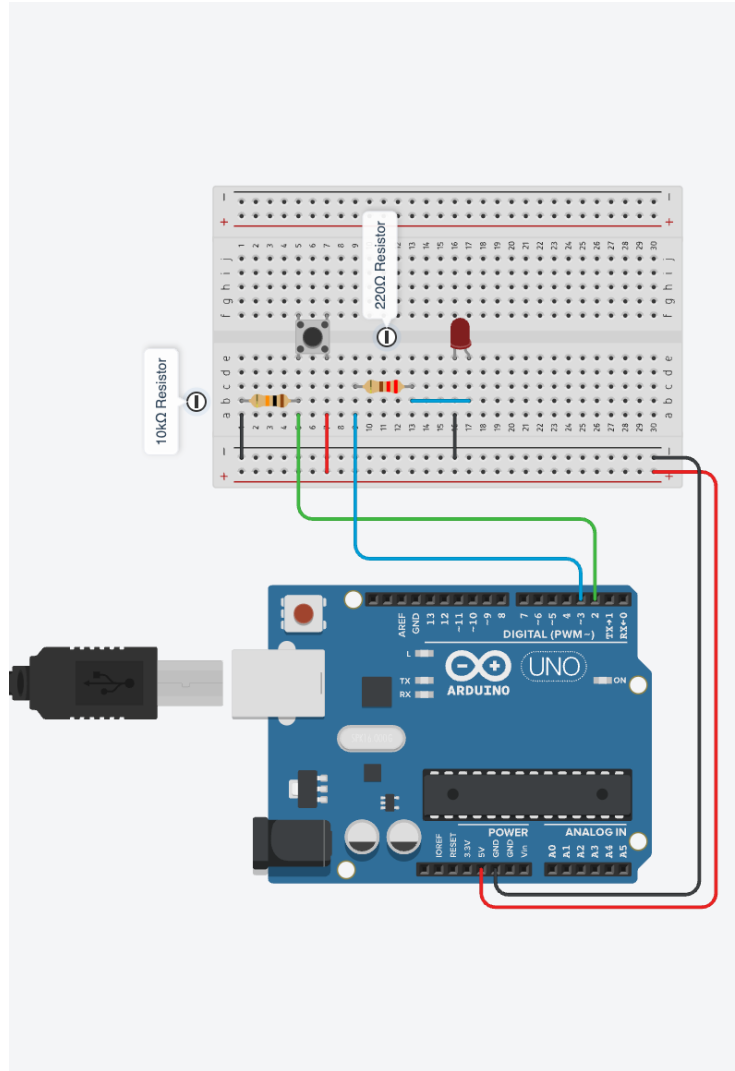


Figure 9: Sample Layout for modified LED circuit

Create Your Project (Sketch)

Now you're ready to create the project (aka sketch) and write the code to control your system. Follow these steps:

1. Start the Arduino Integrated Development Environment (IDE)
2. Go to **File > New** to create a new sketch (project). This will open a blank window where you can begin coding for the Arduino Uno (see Figure 10).
3. Click **File > Save As** and name your project (i.e. *learningArduino.ino*). It's a good habit to avoid spaces or special characters in the file name.

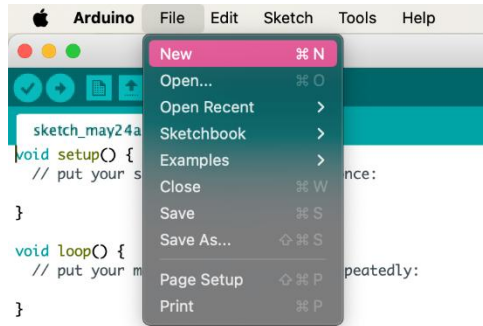


Figure 10: Creating a new project

Add your Code

Now it's time to write your code. We've written it for you this time, so please copy and paste this code into the Arduino IDE:

```
-----
const int buttonPin = 2;
const int ledPin = 3;

// program starts here (function runs once)
void setup () {
  pinMode(buttonPin, INPUT);
  pinMode(ledPin, OUTPUT);
}

// main functional block (runs repeatedly)
void loop() {
  int state = digitalRead(buttonPin);
  if (state == HIGH) {
    digitalWrite(ledPin, HIGH);
  } else {
    digitalWrite(ledPin, LOW);
  }
}
-----
```

Here's an explanation of the code:

1. `const int buttonPin = 2;`
 - a. Declares a constant variable for the button input pin. This means digital pin 2 will be used to detect the state of the pushbutton.
2. `const int ledPin = 3;`
 - a. Declares a constant variable for the LED output pin. This sets digital pin 3 as the one that controls the LED.

3. `pinMode(buttonPin, INPUT);`
 - a. Configures pin 2 as a digital input, allowing it to detect if the button is pressed or not.
4. `pinMode(ledPin, OUTPUT);`
 - a. Configures pin 3 as a digital output, which can be set HIGH (5V) or LOW (0V) to control the LED.
5. `digitalRead(buttonPin);`
 - a. Checks the voltage level of pin 2. If the button is pressed, it reads HIGH; if not pressed, it reads LOW.
 - b. If the button is pressed, `digitalWrite(ledPin, HIGH);` turns the LED on.
 - c. If the button is not pressed, `digitalWrite(ledPin, LOW);` turns the LED off.

Test your Circuit

Upload your code to the Arduino using the upload button (right arrow at top left of IDE)

The system is working correctly if:

1. The LED turns on when the pushbutton is pressed
2. The LED turns off when the pushbutton is released

You can test this by pressing and holding the button — the LED should remain lit for as long as the button is held down. When you release the button, the LED should turn off immediately. If this behavior does not occur, double-check the wiring of your button, LED, resistor, and the code logic. Ensure that the button pin is properly pulled down using a resistor (typically 10k Ω) to avoid a floating input.

Part 3 – Analog Inputs and UART

Analog signals are everywhere in the real world, like temperature, light, or sound. Microcontrollers cannot process these signals directly without an Analog-to-Digital Converter (ADC). Learning to work with ADCs is crucial because sensors that detect analog signals are key in robotics, automation, and even biomedical applications. Understanding how to read analog signals and convert them into data that a microcontroller can use opens possibilities for building complex, responsive systems.

Outcomes

You should be able to do the following things by the end of this section:

- Read and display analog data

Background

Analog Signals

Digital signals can take on 2 different states or voltage levels, high (V_{cc}) or low (GND).

Analog signals, in contrast, can take on any voltage between its minimum and maximum.

Measuring Analog Signals using Microcontrollers – ADCs

Microcontrollers are digital devices, which makes it somewhat complicated to read analog signals. Specialized devices called ADCs (Analog-to-Digital Converters) are used to convert analog voltages into digital values that microcontrollers can understand. The Arduino Uno has built-in 10-bit ADCs on its analog pins (A0–A5), making this process straightforward. This means each analog input is represented by one of 2^{10} or 1024 distinct values where 0 represents 0 V and 1023 represents 5 V.

Potentiometers

A potentiometer acts as a variable voltage divider. The outer pins are connected to a voltage supply and ground, while the middle pin outputs a voltage between these two values, depending on the position of the dial. The output voltage varies linearly with the dial's angle, ranging from 0 V (ground) to the supplied voltage (5 V in this case).

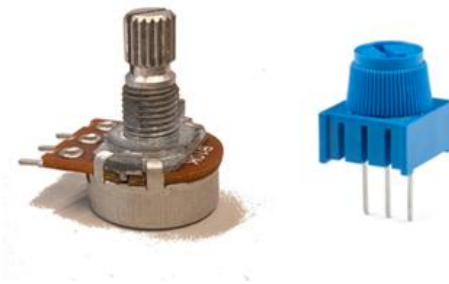


Figure 11 - Potentiometer

UART and Serial Communication

UART (Universal Asynchronous Receiver/Transmitter) is an important protocol for data exchange between devices. Many embedded systems require communication between the microcontroller and a computer or another microcontroller. Mastering UART is necessary for creating systems that can share data, like sensors sending information to a central processor or logging data to a computer.

The Arduino Uno has two dedicated UART pins:

- D0 – RX (Receive)
- D1 – TX (Transmit)

It also has a built-in USB-to-Serial chip. When you connect your Arduino to your computer via USB and use the **Serial.begin()** and **Serial.println()** functions, you're communicating through these UART pins. There's no need to connect anything to the physical pins.

Activity 3 – Read and Display Analog Values

In this task, you will connect the 5V pin to a potentiometer, read its voltage using an analog input pin, and display the value in the Serial Monitor.

Wire your Circuit

Refer to Figure 12 for the circuit's wiring.

1. Use the breadboard's power and ground rails to connect the outer pins of the potentiometer. Note that the orientation of the potentiometer doesn't matter, as it will function if power and ground are connected to the outer pins.
2. Connect the power and ground rails to 5V and GND on the Arduino.
3. Connect the middle (wiper) pin of the potentiometer to analog pin A0.

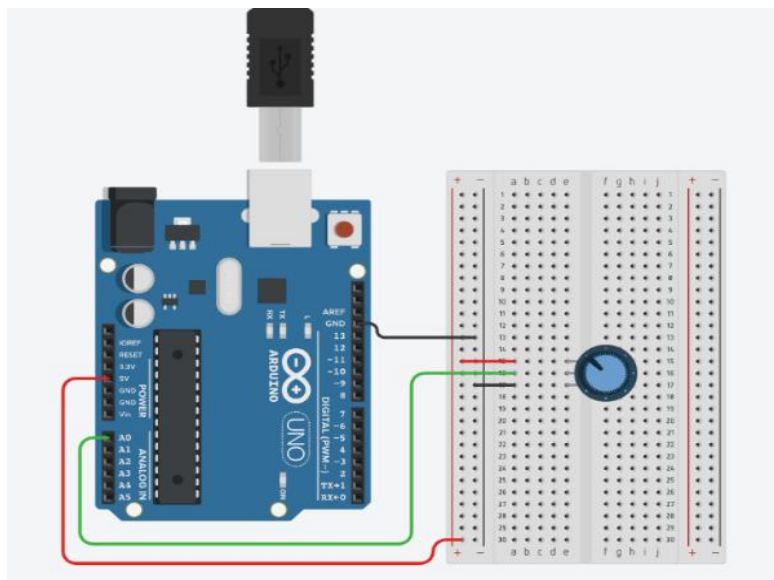


Figure 12: Layout Diagram for Potentiometer

Add your Code

Once again, the code is provided for you. There is no special configuration is required - just open the Arduino IDE and paste this code into a new sketch/project:

```
-----  
int potPin = A0;           // Potentiometer connected to analog pin A0  
int adcValue = 0;         // Variable to store the ADC reading  
  
void setup() {  
  Serial.begin(9600);      // Start serial communication at 9600 baud  
}  
  
void loop() {  
  adcValue = analogRead(potPin); // Read analog voltage (0-1023)  
  Serial.println(adcValue);      // Print the value to Serial Monitor  
  delay(100);                   // Small delay to make output readable  
}  
-----
```

This code reads a variable voltage between 0V and 5V using analog input pin A0.

1. **analogRead(potPin)** reads the voltage on analog pin A0 and converts it to a 10-bit digital value (0 -1023)
2. **Serial.begin(9600)** starts serial communication so you can view the readings in the Serial Monitor
3. **Serial.println(adcValue)** prints the result to your computer

Test your Circuit

Upload your code to the Arduino using the upload button (right arrow at top left of IDE)

To access the serial monitor:

1. In the top left, select **Tools > Serial Monitor**
2. In the top right, select the magnifying glass icon

The system is working correctly if:

1. The analog value read from the potentiometer is displayed in the IDE
2. The displayed analog value changes as you turn the potentiometer's knob